

Trabalho de Implementação – versão 1 – Valor: 10%

• O prazo de entrega é 30/06/2026 às 23:59 horas, com entrega via Moodle.

• O trabalho é de dupla, com apresentação em sala de aula.

Os objetivos deste trabalho são abordar os seguintes tópicos:

• **Threads:**

- Entender como criar e terminar *threads* usando `pthread_create()` e `pthread_exit()`. Dependendo de como você projetar a aplicação, pode haver o uso de `pthread_join()` e `pthread_yield()`;
- Entender como fazer *Inter Process Communication* (IPC) entre as *threads*;
- Entender como sincronizar as *threads* de forma que se evitem condições de corrida, *deadlocks* ou *starvations*;
- Entender a analogia do *buffer* como uma caixa d'água furada com M produtores e N consumidores;
- Entender como se aplicam *mutexes* (*Mutual Exclusions*) e semáforos usando `pthread_mutex_t` e `sem_t`.

• **Aplicações distribuídas com arquitetura Cliente-Servidor:**

- Entender parcialmente como se faz uma aplicação que se comunica com outra via rede de computadores utilizando sockets e como se estabelece uma conexão TCP do ponto de vista do cliente e do servidor;
- Como enviar e receber dados;
- Como o servidor espera por novas conexões.

Para atender aos objetivos, o aluno deve implementar um serviço de chat multicliente via Internet. Sobre isso, do que é composto este trabalho?

1. Servidor – feito em linguagem C:

a. O servidor deve ser capaz de esperar em uma porta TCP especificada pelo usuário por novas conexões proveniente de clientes conectados na mesma rede local. A partir do ponto em que uma nova conexão é aceita, o servidor deve tratá-la via threads. De maneira específica, uma ou mais threads devem ser criadas para cada cliente conectado. Ainda, enquanto o cliente estiver conectado ao servidor, as threads devem existir. No evento de desconexão voluntária de um cliente, as threads relativas a ele devem ser terminadas.

b. Para auxiliá-lo, na página 04 há um diagrama de sequência ilustrando a comunicação entre clientes e servidor em função do tempo. Contudo, uma explicação sobre o protocolo é:

c. Ao aceitar uma conexão de um cliente, a primeira coisa que o servidor deve fazer é enviar uma mensagem de boas-vindas usando *strings* com o seguinte formato: “bom|msg_servidor|Olá! Seja bem-vindo!|eom”;

d. Após isso, o servidor deve ser capaz de receber comandos provenientes do cliente, mas a primeira delas deve ser:

“bom|usuario_entra|apelido_que_o_usuario_digitou|eom”

e. Depois desse comando de conexão, o servidor deve receber comandos dos usuários, como:

“bom|usuario_sai|apelido_que_o_usuario_digitou|eom”

Claramente, o usuário se desconectou da sala de conversa. Outro exemplo é:

f. “bom|msg_cliente|mensagem_que_o_usuario_enviou|eom”

g. É importante ressaltar que cada um dos comandos listados acima (*usuario_entra*, *usuario_sai*, *msg_cliente*) deve gerar uma string que será colocada em um buffer de mensagens que devem ser enviadas para todos os clientes conectados, exemplo:

usuario_entra → “Hugo entrou na sala de conversa.”

usuario_sai → “Hugo saiu da sala de conversa.”

msg_cliente → “Hugo enviou: Sistemas Operacionais é muito massa!”

Disciplina de Sistemas Operacionais

Observe que o comando de enviar mensagem de Hugo foi: `"bom|msg_cliente|Sistemas Operacionais é muito massa!|eom"`. Sendo assim o apelido de cada usuário conectado deve estar armazenado no servidor!

O comando desconectar deve ser tratado de forma que devem ser terminadas todas as threads criadas para aquele cliente.

h. Sempre que houver mensagens no buffer uma ou mais threads devem enviá-las para os clientes conectados usando a seguinte string: `"bom|msg_servidor|string_contida_no_buffer|eom"`;

i. Lembre-se que o acesso ao buffer é algo que deve ser sincronizado. Então, colocar e retirar itens do buffer deve ser implementado usando semáforos e mutexes. Durante o desenvolvimento do seu trabalho, pode haver a necessidade (ou não) de outras regiões exclusivas para utilizar mutex.

2. **Cliente:**

- a. O cliente deve solicitar ao usuário o endereço IP do servidor e a respectiva porta em que se deve conectar;
- b. Deve mostrar a mensagem de boas-vindas;
- c. O cliente deve ler do usuário o apelido que ele deseja utilizar na sala de chat e enviar o comando correspondente de conexão ao servidor e informando o apelido de usuário;
- d. Depois desse momento, o cliente deve gerar uma thread que recebe comandos do servidor e mostra na tela as mensagens que o servidor enviou para o cliente;
- e. Ainda nesse momento, o cliente pode digitar qualquer mensagem que deve ser enviada ao servidor. É importante ressaltar que se o usuário digitar "tchau", o cliente deve enviar o comando de desconexão para o servidor e sair do programa.

3. **Entregáveis:**

- a. **Todos os códigos-fontes do trabalho do cliente e do servidor, tanto para quem faz apenas a disciplina de Sistemas Operacionais quanto para quem faz as duas disciplinas. Assume-se que o compilador utilizado é o gcc. Os códigos não sofrerão nenhuma alteração por minha parte para ser compilado. Caso não seja possível a compilação do código, o trabalho será zerado.**
- b. **Os arquivos executáveis do servidor e do cliente.**
- c. **Arquivo-texto contendo o nome do aluno, assim como os comandos de compilação e de uso do servidor e do cliente.**

Além do arquivo deste enunciado, há três outros arquivos contendo códigos-fonte exemplos que irão auxiliá-los na parte inicial do trabalho. A primeira linha dos códigos-fonte contém um comentário para compila-lo e executar o arquivo-objeto resultante. Para cada um dos quatro códigos-fonte, eu adicionei abaixo perguntas para guiá-lo no processo entendimento:

`prodCons_thread_semaforo.cpp`:

- Como são criadas as duas threads? E se você não souber quantas threads serão criadas, como tratar?
- Qual a função de `pthread_join()` em 52 e 53?
- Como são inicializados e terminados os semáforos? Para que servem `sem_wait()` e `sem_post()` em 16 e 22 e em 33 e 39?
- Onde estão as regiões críticas no código-fonte?

`servidor.cpp`:

- A partir de que linha nós podemos dizer que a conexão do cliente com o servidor está pronta para enviar e receber dados?
- Como os dados são enviados de volta para o cliente?
- Como as mensagens do cliente recebidas?
- Onde a conexão é fechada?

`cliente.cpp`:

- Onde é realizada a conexão TCP com o servidor?
- Como se conectar a um servidor remoto?
- Como os dados são enviados para o servidor e recebidos do servidor?

• Onde a conexão é fechada?

Diagrama de sequência da interação entre servidor e clientes:

